Description thesis

Introduction

Context of digital music production

The use of sound recordings in music is a common practice in current-day digital music production. Typically, such samples are made in acoustic settings and attempt to record a certain source, i.e. an instrument or voice. Acoustic recordings. Recordings that are made outside of such acoustic conditions, and have no specific target source, constitute field recordings or soundscapes. A third general type of recording, is a found sound. Found sounds, like found objects, *objet trouve*, refer to objects that are not typically considered material for art. Simply put, it is any object that has some purpose, art-related or not. In music composition the usage of found sounds, i.e. audio recordings, was first introduced by Pierre Shaeffer (in the 40s) in *Musique Concrete*. *Musique Concrete* utilizes found sounds and digitally synthesized sounds. Digital Signal Processing (DSP) methods can also be used. It is a free form of sound creation which can deviate from the norms of composition (deviation from harmony, rhythm, and melody) [cite fundamentals of dsp, or, Oxford dictionary of music]. Besides Shaeffer there are numerous musicians that have used found sounds in their compositions [Four Tet, Radiohead, Fredagain]. Field recordings, a sub-category of found sounds [cite this], have been used in soundscape composition as a means to communicate the need for soundscape awareness (sound pollution). Can refer to WSP sound collection of Vancouver. Barry Truax, utilized field recordings in his soundscape compositions. His methods utilized derivates of granular synthesis to compose soundscapes. [Ref his ecology based outputs using automata]. Recordings with mobile devices are extremely accessible, which provides computer musicians ample opportunity to work with diverse sound materials. Such as in Truax' compositions, the use of granular synthesis with sample inputs, also Granulation, has been explored thoroughly by Miranda, Roads, Truax himself, and Xenaxis. Xenaxis was the first to propose the use of stochastic and game-theory inspired composition, as an adversary to traditional serial music composition. (In the 50s / 60s) He proposed *Markovian Stochastic Music*, with which he composed *Analogique A* and *Analogique B*, to control grain parametres such as loudness, pitch, and density within sequences of screens. Roads defines grains as microsound events typically lasting anywhere between 1-100ms. Truax also explored the use of probability distributions in his early works with the POD4, POD5, and POD6, which all use poisson distributions to determine sound placements (in the 70s). More recently, the use of DSP methods to analyze grains and allow for interactive grain exploration and granular synthesis has been explored by Diemo Schwarz. With the CataRT project, grains were obtained from large corpora of sounds (find databases used). These were then analyzed using *n* MPEG-7 standard audio descriptors. An interactive interface, CataRT, was then proposed to explore the descriptor space of these grains. Other works have proposed a neural approach to granular synthesis with input sample data. Sounds obtained from three databases were encoded into a latent space. This learned latent space was then used to interpolate between different points in the latent space. This allowed for the reproduction of input sounds and the production of novel sounds. Both the CataRT project and the Neural Granular Synthesis project mainly explored a sub-category of granular synthesis, concatenative synthesis, as a proposed method of synthesis.

This paper will provide a framework for found sound extraction, sample analysis, and granular synthesis. Different methods of granular synthesis are provided in the framework, including grainlet synthesis, single stream concatenative synthesis, and *Markovian Stochastic* synthesis. Librosa, AudioFlux, and Soundfile are the main libraries used for sound extraction and analysis. Specifically, Librosa is used for its display functions and

descriptors for initial prototyping. AudioFlux is used for sound analysis, loading and saving of audio data. Soundfile for loading and saving of data as well. Other important libraries are NumPy, Scikit-learn, and Scipy. NumPy is used to work with the audio arrays. Scipy and sklearn are used for data analysis.

The contribution of this paper is a framework that enables sound object creation from found sounds in the context explained earlier, using classical DSP methods and stochastic models of granular synthesis for grain parametre control and high level composition.

OLD TEXT

Field recordings are recordings of audio that are not recorded in an acoustic setting; the content of a field recording can be of any sound: wind, rain, birds, a piano in a station, a phone-recorded acapella, the list goes on. Field recordings are accessible to nearly everyone. In computer music production field recordings are often used as background ambient noise and are not the centrepiece of a song (ref Olafur Arnalds). As field recordings are often longer, they also comprise various sound events. It is then more difficult to extract the totality of the sound character from a lengthy field recording in a song when it is not simply used as background ambient noise.

There has been recent research into various ways in which input audio, such as textural ambient sound, is encoded into a latent space, and the latent space can be decoded from, allowing for smoother and potentially novel (re)synthesis of sounds. There has also been research into using sampled sounds and large corpora of sounds for musical sound synthesis: concatenative synthesis (the concat, CataRT, Neural GS). Neural latent spaces of sounds where any point in the latent space can be reconstructed into a sound is an example (irish paper/ neural GS paper/ IRCAM papers). Another example is by engel et al. that managed to output a real-time DDSP framework. However, neural methods of sound generation with sample inputs are limited in two main ways: first, outputs are biased to their training data which limits the novelty of the scope of sound generation, and second, the linkage of a sample input to synthesis parametres as in a DDSP framework loses out on much of the information that is available in a more lengthy field recording (e.g. a 1 minute recording of a metro station that is converted to parametres for a synthesis module loses out on much of the nuance of information from the original input). Methods that use the original input data in their output avoid the abstractness problem. The novelty issue is solved dependent of the parametres of such methods. An example of such a method is granular synthesis: the arranging of microsounds, either sampled from an input sound, or granulation as in Microsound (2001, Roads), or from a generated waveform, e.g. a simple sinusoid of a certain frequency, phase, amplitude. One branch of models of granular synthesis is algorithmic modelling of the synthesis parametres. Xenaxis proposed a markovian stochastic approach to modelling grains of sounds and their parametres in a time sequence in his book Formalized Music as an attempt to provide an alternative approach to serial music composition. The stochastic algorithmic approach leverages sampling grains and their parametres from probability distributions and resolves the complexity issue of controlling parametres for many grains. Depending on the probability distributions and its parametres, such as the average rate lambda in a poisson distribution, it should take longer before a time sequence composed by such an algorithm becomes ergodic, stable. The stochastic models allow us to create novel time sequences of grains. This paper proposes an approach in python to algorithmic music which leverages the characteristics of field recordings to be musical centrepieces.

- usage of environmental soundscapes or found sounds in music composition.
- Sample based synthesis methods, computationally expensive methods, data hungry methods.
- Refer to CataRT (IRCAM), the concatenator, roads, xenaxis, miranda, to provide context of this research.

- Refer to DSP methods/libraries?

The problem

- Field recordings are often not used as main sound material in computer music composition, and are typically utilized as ambient noise (find ref)
- Field recordings can be very noisy, vary in content, be lengthy, lower audio quality: difficult to work with
- Music production software is expensive, and has a steep learning curve

The objective

- Provide a framework to generate music from field recording input via granular synthesis
- The framework utilizes classic DSP analysis methods, that utilize the magnitude spectrogram/ FFT to analyze the field recording and its grains
- Introducing granular synthesis as a technqiue to use sample sounds to make music
    - Corpus based granular synthesis
    - Parrticle synthesis
    - Clouds of particles and organised sotchastically/deterministic
- The framework uses markov chains to model state transitions where each state corresponds to a short time window with n grains and m grain parametres
- Refer to DSP methods/libraries?

Contribution

- The framework allows to explore various abstract models to control granular parametres, it allows for leveraging the power classical DSP analysis methods for any input, it outputs audio samples which can be used to compose further in a DAW environment

Background

Fourier Transform

STFT

Spectrogram

Other

DSP Acoustic descriptors

To Cite: Peeters et al (2001), Klapuri et al (DSP for msuic transcription, 2006)

Note: X(k) refers to a vector of RMS levels of sub bands or a DFT spectrum at a time t (a certain frame)

Spectral Features

- Spectral Centroid The sum of normalized magnitude spectrums for each frequency bin.
- Spectral Spread The spectral spread relates to the bandwidth of the frequency spectrum at a frame
-

The following features: *spectral flatness*, *spectral rolloff*, *spectral flux*, and *zero crossing rate*, are effective in discriminating between different instruments (Klapuri et al. Ch 6).

- Spectral flatness  Spectral Flatness Formula Describes how flat the spectrum of a sound is. The flatter the sound, a high value, is a noisy sound whereas the peakier the sound, a low value, is a tonal sound. Ratio of geometric mean to arithmetic mean of an analysis frame.
- Spectral Flux Frame by frame spectral change by comparing the distance between two spectrums at t and t-1. It is defined as the squared l^2 norm of the this frame by frame magnitude difference.
- Spectral Rolloff  Spectral Rolloff Formula Denotes below which frequency bin a certain fraction of the frequency content (γ) resides. Typically, 0.85 and 0.95 are used for γ.
- Zero Crossing Rate  Zero Crossing Rate Formula Is describecfd as the number of times a signal changes sign. It is strongly correlated to the spectral centroid. It describes how much high-frequency content a signal contains. It is effective at discriminating different classes of percussive instruments.

Temporal features

Temporal features are not used in the grain analysis. Temporal features are often used to describe things about the amplitude envelope of signals. However, since the grains sampled in this paper are rather short (100ms) and grains are arbitrarily indexed, temporal descriptors are less valuable. E.g. if we break a sound event by taking an arbitrary micro acoustic event within it, we lose the valuable informatoin that an amplitude envelope tells us about the sound event.

Granular Synthesis

A grain is an atomic sound event, typically having a duration of 1-100ms. A grain has numerous parametres.Some general parametres prevalent in most methods of Granular Synthesis (GS) are grain size duration, grain starting point, grain waveform, stereo position, and amplitude envelope. Depending on the specific organization strategy of the grains, different parametres are most relevant. The main organization methods of granular synthesis as proposed by Curtis Roads in his book Microsound (2001) can be subdivided into two distinct branches: deterministic and non-deterministic methods. Deterministic methods of granular synthesis, such as Synchronous Granular Synthesis (SGS), Pitch-Synchronous Granular Synthesis, or Physical Models of granular synthesis, rely on deterministic global or local specifications of grain parametres. I.e. the model must either individually for each grain, or algorithmically for the set of grains, determine all the grain parametres. On the other hand, non-deterministic methods include stochastic variants, such as Quasi-SGS and Asynchronous Granular Synthesis, and, chaotic algorithms, such as a Lorentz System or a Logistic Map. Chaotic functions are deterministic in the sense that any given input will always yield the same output, however, as these systems are very sensitive to initial parametre conditions, the behaviour and output of these systems we cannot reliably predict. For this reason chaotic functions are included in the latter branch.

Granulation is the use of a sampled sound as input for the grain waveform parametre. Instead of using a wavelet, or some other waveform, each waveform (1-100ms) is taken from this input. Due to the nature of microsounds, a lot of perceptual qualities are lost with lengths shorter than about 40 ms (Roads, 2001).

- What are the three algorithms, what is the previous implementation of these algorithms in music generation/ gs
    - Markov chains: transition probability matrices (Xenaxis, Miranda)
- Background on algorithms

Markov theory

A Markov process or chain constitutes a chain of events by which the next state or event is determined only by the current state (Markov Property). Higher order Markov chains look back a higher number of states to determine the next state. A state j is accessible i -> j if pn_ij > 0 for some n (where n is the number of steps). We assume every state is accessible from itself (p0: 0 steps, p0_ii = 1). Two states can communicate if they can access one another.

In discrete-time Markov chains the time spent in one state is one time unit (1) if a state has no self-transition. It is a geometric random variable otherwise, defined as geometric(1-p_ii). In continuous-time Markov chains the time spent in each state is a continuous random variable.

- Xenaxis Markovian Stochastic Music he outlines different matrical representations of transition probabilities, denoted by two parametres for each grain parametre. I.e., for a state f0, there are two TPMs by which the transition of f0 to f1, ..., fn is determined. Which of the different TPMs is chosen depends on a grain parametre couple defined previously. Xenaxis mentions perturbations to the transition matrix, these perturbations ensure the sound eventually reaches an equilibrium. Brief explanation, he creates a coupling of parametre states to other parametre probability matrices, this way,

TODO section

TODO:

1. check microsound for part about frequency of synchronous synthesis streams where the density of grains are very high, formants, side bands, etc. Also about amplitude of new waveform being increased/ decreased. In the experiments with certain densities
2. Check fundams. of music processing for theory on fundamental freq/ pitch multiples, relative frequency

TODO (week april 19th): read MPEG-7 Paper, read Caterpillar paper, add notes, read through fundamentals of Markov Processes/ Markov transition matrix/ Markov Chains >> Read Markov generative music papers (or GS paper) / implement different modes of classifying grains, i.e. asserting grain states to transition to- and from. Also, consider if another GS method might be more suited for exploring interesting temporal structures in combi. Check datasets used in Neurogranular synthesis paper, The Concatenator / Let it Bee papers (or other papers in the field). Delve into math of spectral descriptors

Methodology

Field recording input

A field recording is recorded using a Google Pixel 9. Typically, field recordings with a length of 10 seconds to 10 minutes are considered. Audio is recorded in mono with a sampling rate of 48kHz and stored initially in .m4a format. The audio is converted to .wav format using ffmpeg in the command line (this process can be done via the programming language of choice). The .wav format audio is loaded with the same sampling rate of 48kHz in mono using the AudioFlux library. The output is an audio NumPy array corresponding to the sampling rate and original duration of the audio input. By default it is loaded in a single channel.

Input audio analysis/ Grain analysis

The analysis uses classic DSP audio descriptors. To compute these descriptors first a Fourier Transform is computed. AudioFlux uses a Based Fourier Transform class which is similar to the Short-Time Fourier Transform (STFT). Similarly to the STFT, specific parametres are chosen for the Fast Fourier Transform (FFT): window size, the number of frequency bins, and the sliding window. These are all powers of two, which is necessary for optimal computational speed of the FFT.

The size of the FFT window must be chosen depending on the grain duration and the sampling rate. In our case the chosen grain duration is 100ms. The grain consists of $0.1*48000=4800$ samples. To capture sufficient detail per grain we must adjust our FFT size accordingly. A large FFT window results in a high frequency resolution and a low time resolution. A small FFT window results in a low frequency resolution and a high time resolution. These are inversely proportional. If we choose an FFT size of 2048 samples, each frame of the transform will be roughly $2048/48000≈0.043s$ long. By decreasing the window size further we would capture more detail in per frame transitions, however, our frequency resolution would drop down further. Our Nyquist frequency is $48000/2=24kHz$. Typically, the number of frequency bins in an FFT computation is half the FFT size plus 1. The number of bins with a window of 2048 is then 1025. The hop size is typically set to one fourth the size of the fft. This allows to capture sufficient detail considering that we are also applying a Hanning window on each analysis window.

The BFT of the entire input audio is computed with these parametres in mind. Once the BFT is computed, each of the aforementioned spectral features are computed on the BFT. For each analysis window of 43 ms a descriptor value is computed. To fit this to our grains of 100ms duration, the computed descriptor array values for the entire input audio are allocated to each sample. E.g. with an input of 10s duration, there are $10*48000=480,000$ samples. The first 2048 samples are assigned the first descriptor frame value, the next 2048 samples the next descriptor frame value, and so forth. Then the mean over samples is computed to obtain the per grain descriptor values. The result is a table where each row corresponds to a grain; the row contains its unique hashed ID, its sampling rate, its starting index in the loaded audio array, its size in samples, and its descriptor values in each of the following columns. This table is stored in a .csv file.

For visualization of the grains in a lower dimensional space the choice can be made between selecting two or three descriptors and use those columns for analysis, or, to use dimensionality reduction methods. In this paper only Principal Component Analysis is considered. For the synthesis stage of the framework, it is useful to compute clusters of grains based on their spectral features. KMeans is chosen as an unsupervised method to clusters the grains together. The scikit-learn library offers a powerful class and method to fit n_clusters to a set of data. The spectral features exist on different scales, so to ensure the data is scaled before passing it into the KMeans algorithm, the data is first scaled using the StandardScaler object form scikit-learn. The main parametre of interest in the KMeans algorithm is the number of clusters. The choice for the number of clusters should be made depending on the visual analysis of the grains and the desired size of the state space later on in the pipeline. Typically, when experimenting with different parametre sizes n_clusters=3 has been a viable choice for different field recording inputs.

Markov Granular synthesis

With the markov chain granular synthesis methods the grain parametre control is managed by a transition probability matrix (TPM). The probability distributions for the rows of the matrix, i.e. the state transition probabilities, can be determined in different ways. Methods of TPM intialization explored in this research include frequency based probability estimation, uniform random sampled probability distributions, and manually input transition probabilities. Also particular to this approach is that we must designate states for the state transitions modeled by the TPM.

Transition Probability Matrices and Markov States

A state in the context of this research is a vector of grain parametres. The grain parametres selected in this research were grain density over a time window Δt, grain size, and grain cluster. The density values can be chosen arbitrarily, so long as they are a non-negative integer. The grain size values are also selected arbitrarily; however, one should consider that sampled grains shorter than 40ms become nearly undiscernable from one another, and grains longer than 100 ms are atypical of granular synthesizer, as grains length range from 1ms to 100ms, however longer grain durations are certainly viable. Finally, the grain clusters specify the cluster from which the grain waveform parametre will be sampled. The sampling from a cluster is with uniform probability.

The TPMs are then either generated for each separate grain parametre, or the TPM for the vector parametre states is computed at once. With the prior approach, you can determine via one of the three mentioned approaches how to initialize the TPM for that specific parametre. It should be noted that the frequency sampling is only viable for the grain cluster parametre since we consider no data from which we could sample density and size parametres. To compute the frequency based probability distribution, the original input audio is sampled from. Iteratively, a window passes over the input, the window duration should correspond to the grain duration set previously and counts how the frequency of a certain grain type (a grain belonging to a cluster n) is followed by another grain type. Then the row of frequencies is divided by the sum of the row, resulting in probabilities that sum to 1 (normalization). For the uniform random initialization the initial values are selected stochastically, but to ensure each row sums to 1 the rows are also divided by their sum total. This normalization is only necessary for the manual input approach if the matrix values are not selected to sum to 1. To compute a full TPM for the entire state space if individual parametre TPMs were computed, the Kronecker product of the matrices are taken, which we do because we assume the random variables determining the parametres values are independent from one another.

Synthesis parametres

The main parametres other than the transition probabilities and grain parametres will be elaborated here. Firstly, the number of grains *n_grains* is decided, typically in my experiments values were selected between 1 and 4 grains. Higher grain values are possible, however, the more grains are selected, the greater the loss of information from individual grains. This is due to the fact that the inherent loudness of grains is not controlled. If one grain has a high loudness value, the output for that time window will be dominated by the loud grain. Attempts to separate grains in stereo are made in the linked parametre section. The time window (or screen duration) is denoted by the paramere Δt. Typically, a value between 100ms and 1000ms was chosen. With exploration of values below a 100ms, it should be noted that grain waveform information will be lost in the output as the grain duration is clipped to the length of the window. Higher densities in short time windows will also result in grain overlap and the density of the grain will produce frequency bands and potentially sidebands (Microsound Ch3, and for this provide specific moment analsysis in such an output) corresponding to the grain density parametre. Finally, the last parametre for the synthesis is the number of output channels *num_channels*. As two channels are sufficient for producing stereo effects, which is explored in the Linked Parametres section, only two channel outputs are used.

Initial states

The initial grain states are determined either manually or uniform randomly for each grain $g_1, g_2, ..., g_n$. The number of initial states corresponds to the number of grains n_grains.

Linked parametres

As in grainlet synthesis, parametre linkage has also been explored in this module. Stereo shifting was applied such that the first indexed density d parametre value is linked to a stereo shifting of 0, i.e. no stereo shifting. Each next parametre is linked so that the right channel, of the two output channels, is shifted by a multiple of 1ms. For instance, the second density d is would be shifted an non-negative integer multiple $n * 48$ when our sampling rate is set to 48kHz.

Output

The output is obtained by intializing an empty array, corresponding to the number of channels. For each iteration a buffer array is created corresponding to Δt of 0 valued entries. Then, for each i_th grain its information is added to this buffer. Each buffer is then concatenated with the output buffer. The output is an array of shape (*num_channels*, *n_iterations * sampling_rate * Δt*). The soundfile python library is used to save the data to .wav format. Along with the audio data, all the described parametres above, aside from the TPMs, are saved to JSON files.

Evaluation

To evaluate the output and the entire framework multiple methods are available.

Notes:

- Design science:
    - Keep track and log production process
    - Note moments of creative production - link to some output
    - Document failures

Discussion

- Dynamic analysis of which descriptors for a given input. I.e. which descriptor vector best discriminates between all the different sound events present in the input. Again, for which set of descriptors can similar grains have low variance, and different grains have high variance.
- Grain size as parametre is means that the initial analysis performed on the set grain duration is thrown out of the window in a sense.

Questions

Q: since grain slicing/ selection is interwoven to a significant degree in a given algorithm, e.g. Markov chain of grains includes the selection. This depends on what granular parametres are handed off to the algorithm. It makes more sense to explore the workflow/ output of the three algorithms given three different types of grain analysis. Instead of having the workflow: 1) 3? grain slicing, 2) 3? grain selection, and 3) 3 grain synthesis, you would have 1) 1 grain slicing method, 2) 3 grain analysis representations: MFCC, Spectral descriptors, latent (RAVE Embedding for example), and 3) 3 algorithms that leverage the different analyses of the different representations.

Q: RQ - Can markov chains propose a viable algorithmic method to granular synthesis?

Meeting 21 april notes:

Meeting 28 april notes:

- byte tracking (RoboFlow, already implemented)

Meeting 12 May notes; What I did:

1. Descriptors and literature
2. Pipeline is working
3.

To discuss:

- poster
- may: write paper, do more experiments, develop more markov approaches.
- currently: Design Approach, log experiments with different inputs, different functions,
- markov experiments,
- analysis update,

Ideas

1. use descriptor values in grain parametre setting/ probability distribution coupling.
2. stat analysis between different hop window sizes

method/ bg text?

The larger the window, the greater our frequency resolution and th. The smaller our window, the The window size is set to 2048 samples. Considering our sample rate of 48kHz, the lowest detectable frequency becomes ≈23.4Hz. This is right about the lower hearing bound of human hearing (≈20Hz). In signal analysis, the time resolution is inversely proportional to the frequency resolution. If we increase our window size, we capture less time snapshots of the evolution of a signal, thus resulting in a lower temporal frequency. With the sampling rate at 48kHz, the window duration is $2048/48000≈0.043$.

Time-varying descriptors are then computed using the Librosa spectral features. For fast computation of the Short-Time Fourier Transform (STFT), the size of the window, the number of bins, and the hop size all need to be powers of two; this is a feature of the Fast Fourier Transform (FFT). The window size is set to 2048 samples. Considering our sample rate of 48kHz, the lowest detectable frequency becomes ≈23.4Hz. This is right about the lower hearing bound of human hearing (≈20Hz). In signal analysis, the time resolution is inversely proportional to the frequency resolution. If we increase our window size, we capture less time snapshots of the evolution of a signal, thus resulting in a lower temporal frequency. With the sampling rate at 48kHz, the window duration is $2048/48000≈0.043$. The input signal is split into windows of 43ms. If we want to increase our time resolution, we must take one power of two less in our window size, thus, 1024 samples, and so forth.

The frequency resolution is determined by the numbers of frequency bins we consider in our window. By the Nyquist frequency we only need frequency components below half of our sampling rate. In this case we need only consider frequencies below 24kHz. Typically, our FFT has the same size as our window, thus, 2048. These can be adjusted to increased time or frequency resolution. The size of our fft, i.e. the number of samples we consider, is thus both linked directly to the duration of our window and the number of frequency bins we can consider. The hop size is set to 512 and determines the distance between one column computed of a signal and the next. The window uses a hanning window to ensure smooth windowing. To ensure all information of a signal is captured in the STFT, the hop length is typically set to $window length//4$, therefore 512. From the STFT we can now compute any of the available descriptors: spectral centroid, rolloff, contrast, rms, and zero-crossing rate.

To associate descriptor values with the grains, the stft frames over which the descriptor values are computed are reformatted to fit the sample size of the original input ($total sample size = sample rate * input duration$). There is now a descriptor value at each sample point, which allows us to take the mean and standard deviation of each sample window corresponding to our grain size. Each grain now has an adhering mean/ std of some descriptor value. As our analysis window is approximately 43ms, and our grains are 100ms, each grain takes a mean of ≈2.3 analysis window values.

We have now managed to compute descriptors for each grain. The current problem is that we want to analyze grains from different inputs. An optimal approach might be

- PCA for dimensionality reduction or visualization
- k-means
- gaussian mixture models
- DBSCAN